

第三模块：RTOS 与任务调度机制（嵌入式装备方向）

— 超详细学习内容

本模块目标：从“单循环程序思维”升级为“多任务实时系统思维”。重点理解任务如何切换、上下文如何保存、优先级为何决定实时性，以及RTOS内部真正做了什么。

A. 基础知识（RTOS核心概念）

A 1 . 什么是RTOS

RTOS (Real-Time Operating System) 的核心不是界面，也不是文件系统，而是 **任务调度器 (Scheduler)** 。

RTOS负责：

- 决定哪个任务运行
- 何时切换任务
- 管理延时与时间片
- 提供同步机制

重要认知：

RTOS并不会让程序运行更快，它的意义是让系统行为“可预测”。

A 2 . 任务 (Task) 的本质

一个任务本质上是：

- 一段函数入口地址
- 一块独立的栈空间
- 一组保存的CPU寄存器

任务切换本质：

保存当前任务上下文 → 恢复下一个任务上下文。

必须理解：

任务不是线程库里的概念，而是 **CPU上下文集合**。

A 3 . 任务状态模型

典型状态：

- Running（正在运行）
- Ready（就绪）
- Blocked（等待事件）
- Suspended（挂起）

关键思想：

优秀设计中，大多数任务应长期处于 Blocked，而不是轮询。

A 4 . 时间基准（Tick）

RTOS通过 SysTick 或硬件定时器产生周期中断。

作用：

- 更新系统时间
- 处理延时任务
- 触发调度检查

注意：

Tick频率过高会增加CPU负担。

B. 调度机制核心原理

B 1 . 抢占式调度

抢占式调度意味着：

高优先级任务可以随时打断低优先级任务。

优点：

- 响应快
- 实时性好

风险：

- 优先级设计不当会导致系统混乱。
-

B 2 . 时间片轮转

当多个任务优先级相同时：

调度器按时间片切换任务。

核心目的：

- 防止某个任务长期占用CPU。
-

B 3 . 上下文切换全过程（必须理解）

一次任务切换通常包括：

- 1) 保存当前任务寄存器到栈
- 2) 保存栈指针到TCB
- 3) 加载下一个任务栈指针
- 4) 恢复寄存器
- 5) 返回执行

你需要理解：

任务切换不是函数调用，而是“CPU状态搬运”。

B 4 . 为什么使用 PendSV 进行任务切换

PendSV 通常被设置为最低优先级。

原因：

- 避免打断高实时中断
- 保证系统行为稳定

核心思想：

调度请求先记录，真正切换延迟到安全时机执行。

C. 同步机制（任务协作基础）

C 1 . 队列（Queue）

用途：

- 任务间数据传递
- ISR 与任务通信

优点：

- 自带阻塞机制
 - 解耦模块
-

C 2 . 信号量 (Semaphore)

两种常见类型：

- 二值信号量
- 计数信号量

使用场景：

- 事件通知
 - 资源计数
-

C 3 . 互斥锁 (Mutex)

作用：

保护共享资源。

关键点：

- 支持优先级继承
 - 防止优先级反转
-

C 4 . 事件组 (Event Group)

特点：

- 多事件组合触发
 - 位掩码同步
-

D. 进阶知识（系统实时性核心）

D 1 . 优先级反转

现象：

高优先级任务等待低优先级任务释放资源，而中优先级任务不断运行。

解决方案：

- 优先级继承机制。
-

D 2 . 临界区与关中断

RTOS中常见做法：

- 进入临界区关闭调度或中断
- 退出临界区恢复

注意：

临界区必须尽量短。

D 3 . 栈管理

每个任务必须分配独立栈。

风险来源：

- 局部变量过大
- 深层函数调用
- ISR嵌套

建议：

开启栈溢出检测。

D 4 . 空闲任务（Idle Task）

作用：

- 回收资源
- 进入低功耗

不要在Idle任务中放业务逻辑。

E. 高频技术考点（标准答案）

E 1 . 为什么任务切换要保存寄存器？

标准答案：

因为每个任务都有独立执行上下文，必须保存当前CPU状态，才能在恢复时继续执行。

E 2 . 为什么 PendSV 优先级最低？

标准答案：

为了避免任务切换打断高实时中断，使调度在安全时机进行。

E 3 . 为什么RTOS任务不应该使用while(1)轮询？

标准答案：

轮询会占用CPU，阻塞其他任务，应使用阻塞机制等待事件。

E 4 . 优先级反转是什么？

标准答案：

高优先级任务被低优先级任务间接阻塞，中间被中优先级任务抢占。

F. 注意事项（工程经验）

- 1) 避免创建过多高优先级任务。
 - 2) 任务栈不要盲目缩小。
 - 3) ISR中尽量使用FromISR接口。
 - 4) 不要在多个任务中频繁printf。
 - 5) 同步机制优先使用队列而不是全局变量。
-

G. 深度练习建议

- 手写一个最小RTOS调度模型（数组+优先级选择）。
- 观察任务切换时栈变化。
- 故意制造优先级反转并观察系统表现。

完成本模块后，你将从“会用RTOS API”提升到“理解调度器内部逻辑”。